

Git e GitHub na infraestrutura

Como o código da equipe chega ao servidor?



"Manda o zip no grupo que eu subo pro servidor."

...três dias depois: *qual versão está no ar mesmo?*

 projeto-final-v2-
AGORA-VAI.zip

 Alguém editou
direto no servidor

 Quem mudou?
Quando? Por quê?

 Quebrou. Como
voltar?

Do código ao usuário: a jornada



ONDE PARAMOS

Servidor **srv-grupo** acessível por SSH, com a **app-turma** no ar na porta **8080**.

HOJE

Trocar a página provisória pelo **código do repositório do grupo** — e atualizar sem bagunça.

Ao final desta aula, você vai...

- 1 Explicar **repositório, commit, push, clone e pull** e onde cada um acontece
- 2 Publicar o código do grupo no **GitHub** a partir do seu PC
- 3 Fazer o **deploy** no servidor com **git clone** e atualizar com **git pull**
- 4 Diagnosticar um **pull que falhou** com git status, diff e log — sem perder nada
- 5 Relacionar o git com a **rede** : DNS, TCP, porta 443 ou 22

"Hoje eu fiz o código do meu grupo chegar ao servidor — e sei exatamente qual versão está no ar."

Git não é GitHub

TEORIA

GIT — a ferramenta

- Programa instalado **na máquina**
- Guarda o **histórico** de versões do projeto
- Funciona **sem internet**
- Está no seu PC e no servidor

GITHUB — o ponto de encontro

- **Servidor na internet** que hospeda repositórios
- Todos da equipe enviam e buscam dali
- Acesso **pela rede**: HTTPS ou SSH
- Alternativas: GitLab, Gitea, Bitbucket

 Analogia da cozinha: o GitHub é o **caderno oficial de receitas**. Cada cozinha tem uma **cópia**. Receita nova? Primeiro vai para o caderno oficial.

Repositório e commit

REPOSITÓRIO

A pasta do projeto + a pasta oculta `.git` com todo o histórico.

COMMIT

Uma **foto** do projeto num momento, com **autor**, **data**, **mensagem** e um **identificador** (hash).

```
git log

commit 889108f3c1e0a47b9d2...
Author: Ana Souza <ana@email.com>
Date: Mon Sep 21 10:14:02 2026

    Adiciona contato do suporte

commit 081a0ed5b7...
Author: Bruno Lima <bruno@email.com>
Date: Mon Sep 21 09:58:40 2026

    Adiciona integrantes
```

hash

o "RG" da versão

autor

quem mudou

data

quando mudou

mensagem

por que mudou

O caminho do código

PC do desenvolvedor

```
edita arquivos  
git add .  
git commit -m "..."  
cria versões localmente
```

git push

→
envia commits



GitHub

fonte da verdade

git pull

→
busca
commits

Servidor

```
git clone # 1 vez  
git pull # sempre  
só recebe código
```

COMANDO	ONDE	O QUE FAZ
<code>clone</code>	servidor / PC	baixa o repositório inteiro pela primeira vez
<code>commit</code>	PC	registra uma versão só na sua máquina
<code>push</code>	PC	envia seus commits para o GitHub
<code>pull</code>	servidor / PC	traz os commits novos e atualiza os arquivos

Publicar o código do grupo

PRÁTICA • ETAPA 1

Terminal do SEU computador

```
# 0. uma vez por PC: quem é você?
git config --global user.name "Ana Souza"
git config --global user.email "ana@email.com"

# 1. no GitHub: New repository → projeto-grupo3
#   Public · marque "Add a README"
git clone https://github.com/usuario/projeto-grupo3.git
cd projeto-grupo3

# 2. copie site-modelo/index.html para esta pasta
#   e troque NOME DO PROJETO e SEU GRUPO AQUI
git status
git add index.html
git commit -m "Primeira versão do site"
git push
```

✓ FUNCIONOU?

Atualize a página do repositório no GitHub: o `index.html` e o seu commit aparecem lá.

🚨 PEDIU SENHA?

O GitHub **não aceita** a senha da conta no terminal. Use o login pelo navegador que o git abre, ou um **token**.

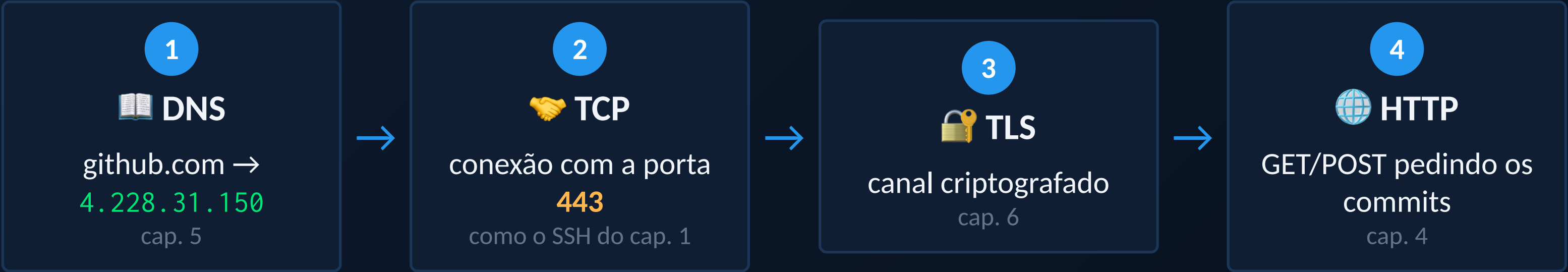
🤔 Rode `git status` antes e depois do commit. O que mudou na mensagem?

Conceito → comando

QUERO SABER...	COMANDO	O QUE PROCURAR NA SAÍDA
Tem algo mudado aqui?	<code>git status</code>	working tree clean ou modified :
O que exatamente mudou?	<code>git diff</code>	linhas com + (entrou) e - (saiu)
Qual versão está aqui?	<code>git log --oneline</code>	o commit do topo é o atual
De onde veio o código?	<code>git remote -v</code>	a URL do origin
O GitHub tem novidade?	<code>git fetch</code> + <code>git status</code>	behind (atrás) · ahead (à frente)
Quais commits faltam?	<code>git log HEAD..origin/main</code>	commits que estão no GitHub e não aqui
Atualizar os arquivos	<code>git pull</code>	Fast-forward ou Already up to date

*Regra do servidor: **olhe antes de mexer**. `status`, `log` e `diff` não alteram nada.*

O que viaja na rede num git clone



URL	PROTOCOLO · PORTA	AUTENTICAÇÃO
https://github.com/u/repo.git	HTTPS · 443	repositório público: ler não precisa de senha
git@github.com:u/repo.git	SSH · 22	chave SSH cadastrada no GitHub

🔑 O servidor só precisa ler o repositório. Dar a ele permissão de escrever seria poder demais → cap. 9.

O servidor também usa git

PRÁTICA • ETAPA 2

Terminal – servidor

```
git --version
cd ~
git clone https://github.com/everonscherrer/\
infra-devops-aula.git
cd infra-devops-aula
ls -la          # achou a pasta .git?
git log --oneline # commits do professor
git remote -v
git status
git pull
Already up to date.
```

📖 Este é o repositório das aulas. Daqui em diante, no início de cada aula:

```
cd ~/infra-devops-aula && git pull
```

🤔 Você consegue dar **push** neste repositório? Por que o **pull** funcionou sem pedir senha?

🤔 **Already up to date** quer dizer que o servidor está sem internet? Como ter certeza?

Espiando a conversa com o GitHub

Terminal – servidor

```
getent hosts github.com
4.228.31.150      github.com

GIT_TRACE_CURL=1 git ls-remote https://github.com/everonscherrer/infra-devops-aula.git \
  2>&1 | grep -E "Connected to|SSL connection|GET /|HTTP/2 200"
== Info: Connected to github.com (4.228.31.150) port 443
== Info: SSL connection using TLS1.3 / ECDHE_RSA_AES_128_GCM_SHA256
=> Send header: GET /everonscherrer/infra-devops-aula.git/info/refs?service=git-upload-pack HTTP/2
<= Recv header: HTTP/2 200
```

getent hosts — o DNS traduziu o nome em IP. É o mesmo IP do trace?

port 443 — o git usou TCP na porta do HTTPS, e não uma "porta do git"

GET /.../info/refs — por baixo, o git faz pedidos HTTP comuns

💡 O IP do GitHub pode ser diferente no seu teste: serviços grandes usam vários endereços. Anote o seu na ficha.

Deploy: o repositório vira produção



Deploy = colocar uma versão do código para funcionar onde os usuários acessam (a **produção**).

⚠ Regra de ouro: o servidor é **cópia**. Mudança de código nasce no PC, passa pelo GitHub e só depois chega aqui.

Missão: primeiro deploy do grupo 🚀

Terminal – servidor

```
cd ~/infra-devops-aula && git pull
sudo bash capitulo-02/lab02.sh preparar https://github.com/usuario/projeto-grupo3.git

cd /srv/app-turma
git log --oneline
ls -l
```



Abra
`http://172.30.0.10:8080` no
seu PC. É a página do grupo?



O **Jorge**, colega da equipe,
também tem acesso a este
servidor e "resolve umas coisas"
quando é urgente.



Mesmo combinado do cap. 1:
não abra o `lab02.sh`. Investigue
com os comandos.

Atualizar a produção

PRÁTICA • ETAPA 4

Terminal do SEU computador

```
# troque "Integrante 1" pelos nomes  
# do grupo no index.html  
git status  
git diff  
git commit -am "Adiciona integrantes"  
git push
```

Terminal – servidor

```
cd /srv/app-turma  
git pull
```

0 esperado...

```
Updating b4aef10..081a0ed  
Fast-forward  
index.html | 4 +++-  
1 file changed, 3 insertions(+), 1 deletion(-)
```

...e o que apareceu

```
Updating b4aef10..081a0ed  
error: Your local changes to the following  
files would be overwritten by merge:  
        index.html  
Please commit your changes or stash them  
before you merge.  
Aborting
```

Sintoma: o pull foi abortado

NO GRUPO DA EQUIPE

"Fiz o push faz 10 minutos e o site continua igual!"

 Pensou em apagar a pasta e clonar de novo? Ou em `git restore`? Pare. Antes, descubra o que você apagaria.

 Só comandos que **olham**: `status` · `diff` · `log` · `fetch` · `ls -l`

Roteiro de investigação

Responda cada pergunta **antes** de corrigir qualquer coisa.

- 1 Segundo o git, o que há de diferente no servidor? → `status`
- 2 O que exatamente foi alterado? → `diff`
- 3 Quando o arquivo foi mexido? Tem pista de quem? → `ls -l` · `diff`
- 4 Essa alteração existe no GitHub? → `fetch` · `log HEAD..origin/main`
- 5 Como atualizar a produção **sem perder** a alteração e **sem editar** no servidor?

Pistas para interpretar o que você vê

VOCÊ VIU...

SIGNIFICA QUE...

`modified: index.html`

o arquivo foi alterado **aqui** e isso não virou commit

`would be overwritten by merge`

o git **se recusou** a apagar trabalho que só existe nesta máquina

`Your branch is behind ... by 1 commit`

o GitHub tem commits que o servidor ainda não aplicou

`Your branch is ahead of ...`

alguém fez **commit no servidor** — isso não está no GitHub

`git restore arquivo`

descarta a alteração local e volta ao último commit ⚠ sem desfazer

`Already up to date`

não há commits novos no GitHub... **ou** ninguém deu push

Mudou no servidor e o navegador mostra o antigo

cache do navegador: **Ctrl+F5**

✅ **ACHA QUE RESOLVEU?**

`sudo bash capitulo-02/lab02.sh verificar`

5 primeiras linhas = **missão**. As outras 3 = **desafio final** →

Entendeu ou só copiou?

- 1 Por que o git **abortou** o pull em vez de sobrescrever o arquivo?
- 2 Se você rodasse **git restore** logo de cara, o que o cliente perderia?
- 3 Se o servidor pegar fogo hoje, o que se perde e o que se recupera? Por quê?
- 4 O servidor precisou de senha para o **pull**? E precisaria para um **push**?
- 5 Que porta e protocolo o **clone** usou? E se a URL fosse **git@github.com:...**?
- 6 Abra **http://172.30.0.10:8080/.git/config**. O que apareceu? Isso é um problema?

As respostas vão para a ficha de deploy e para o AVA.

Deploy em um comando

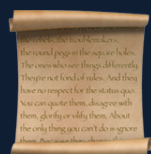
Todo mundo commita



Cada integrante faz **1 commit** do próprio PC e conta

pista: `git log --format='%an'`

Crie o `deploy.sh`



No **PC**, versionado no repositório

pista: o que você digitou para atualizar?

Use só ele



Nova mudança no PC → push →

`bash deploy.sh`

pista: nada de editar no servidor

deploy.sh – esqueleto

```
#!/usr/bin/env bash
# Atualiza a produção com o GitHub
cd _____ || exit 1
git ____ --ff-only
git log -1 --oneline
```

🤔 Para que serve o `--ff-only`? O que ele impede se alguém tiver feito commit no servidor?

🚩 **META**

`lab02.sh verificar` → 8 de 8

O que entregar no AVA



Repositório

Link do repositório do grupo, com o `deploy.sh` e commits de todos.



Ficha de deploy

`ficha-deploy.md` com o diagnóstico e as respostas.



Verificador

Bloco Evidência do `lab02.sh` `verificar`, com 8 de 8.



Uma frase

"Hoje eu consegui ___ e entendi que ___."

 O deploy fica no volume `/srv` do servidor. No Capítulo 3 este repositório recebe a **aplicação de verdade**.

PERGUNTA RÁPIDA

No servidor, `git pull` responde **Already up to date**.

O colega jura que mudou o site e fez o commit. Causa mais provável?

- A O servidor está sem acesso à internet
- B O commit ficou só no PC do colega: faltou o push
- C A porta 8080 está fechada no servidor
- D O git pull só funciona com sudo



Resumo: sua caixa de ferramentas

No PC

```
git clone url  
git add · git commit -m  
git push
```

No servidor

```
git clone url # 1 vez  
git pull --ff-only  
bash deploy.sh
```

Olhar sem mexer

```
git status · git diff  
git log --oneline  
git remote -v
```

Comparar com o GitHub

```
git fetch  
git log HEAD..origin/main
```

Rede

```
getent hosts github.com  
HTTPS 443 · SSH 22
```

Com cuidado

```
git restore arquivo  
# descarta – sem volta
```

 O caminho do código é **sempre**: PC → commit → push → GitHub → pull → servidor. Atalhos nesse caminho viram "funcionava ontem".



O código chega ao servidor.

Mas a aplicação real tem frontend, backend e banco. Como eles conversam?



Próxima aula — Capítulo 3: Docker Compose + Redes

Traga o repositório da aplicação que vocês estão desenvolvendo.